

Distributed Pretraining and Supervised Fine-Tuning of a 1-Billion-Parameter Local Large Language Model with PyTorch FSDP2

By Rui Yang

06/10/2026

Purpose:

The purpose of this lab is to build, train, and deploy a complete large language model (LLM) entirely on local hardware, taking a 1.01-billion-parameter GPT-style transformer from random weights to a working, locally hosted chatbot. Unlike other locally host LLMs, where a pre-trained open-source model was downloaded and hosted through Ollama, this lab covers the full training pipeline itself: tokenizer training, large-scale pretraining distributed across three GPUs with PyTorch FSDP2 (Fully Sharded Data Parallel, version 2), a continued-pretraining "anneal" phase on higher-quality data, supervised fine-tuning (SFT) into an instruction-following chat model, and finally interactive inference with proper decoding controls. The end goal is a private, offline-capable chat assistant where every stage — data, training, and inference — is owned, understood, and run locally, with no data ever leaving the local network.

Background Information:

Large Language Models and the Transformer Architecture:

A large language model is a neural network trained to predict the next token (a word fragment) in a sequence of text. Modern LLMs are built on the transformer architecture, specifically the decoder-only variant popularized by the GPT (Generative Pre-trained Transformer) family. A decoder-only transformer is a stack of identical blocks, each containing a multi-head self-attention layer (which lets every token look back at all previous tokens to gather context) and a feed-forward layer (which transforms each token's representation independently). The model trained in this lab uses 18 transformer blocks with a model dimension of 2048, 16 attention heads, and a feed-forward dimension of 8192, totaling 1,009,047,552 (~1.01 billion) parameters. Instead of learning positional embeddings, the model uses Rotary Positional Embeddings (RoPE), which encode token positions by rotating the query and key vectors inside the attention mechanism — the same technique used by modern models such as Llama. Text is converted to tokens with a custom Byte-Level Byte Pair Encoding (BPE) tokenizer with a vocabulary of 50,257 tokens, trained from scratch on the same data mixture used for pretraining.

The Three Stages of Training:

Production LLMs are not trained in a single pass; they go through distinct stages. Pretraining is the longest and most expensive stage, where the model learns language, facts, and structure by predicting the next token over billions of tokens of broad web, encyclopedia, and book text. The anneal (continued pretraining) stage follows: the learning rate is lowered, and the model is

trained for a shorter period on a smaller, higher-quality data mixture, which measurably sharpens the model without erasing what it already knows. A portion of the original pretraining data is "replayed" during this stage to prevent catastrophic forgetting, which is the tendency of a neural network to lose previously learned knowledge when trained on new data. Finally, supervised fine-tuning (SFT) converts the raw text-completion model into an assistant by training it on prompt/response pairs formatted as conversations. A critical detail of SFT is the loss mask: the loss is computed only on the tokens of the assistant's response, not on the user's prompt, so the model learns to write answers rather than to imitate questions.

Distributed Training and FSDP2:

A 1-billion-parameter model in mixed precision requires far more memory than what its raw weights represent: in addition to the parameters themselves, training must store gradients and two Adam optimizer state tensors per parameter, plus activations. This exceeds the 16 GB of VRAM available on any single GPU used in this lab, which makes distributed training mandatory rather than optional. PyTorch's Fully Sharded Data Parallel (FSDP) solves this by sharding, a process of splitting the parameters, gradients, and optimizer states evenly across all GPUs, so that each GPU permanently holds only one third of the model. During the forward and backward pass, each transformer block's full parameters are temporarily reassembled (all-gathered) on every GPU just long enough to compute, then immediately freed. This lab uses FSDP2, the rewritten API based on the `fully_shard()` function and the `DTensor` (distributed tensor) abstraction, which replaces the original FSDP1 wrapper class. FSDP2 offers per-module sharding, cleaner integration with PyTorch Distributed Checkpoint (DCP) for saving sharded checkpoints without gathering the full model onto one GPU and removes the deprecation warnings associated with FSDP1's `ShardedTensor`. Training also uses `bfloat16` mixed precision (half-size numbers for compute, full precision for the optimizer) and gradient accumulation, where several micro-batches are processed before each optimizer step to simulate a larger batch; gradient synchronization between GPUs is correctly skipped on all but the final micro-step to reduce communication overhead.

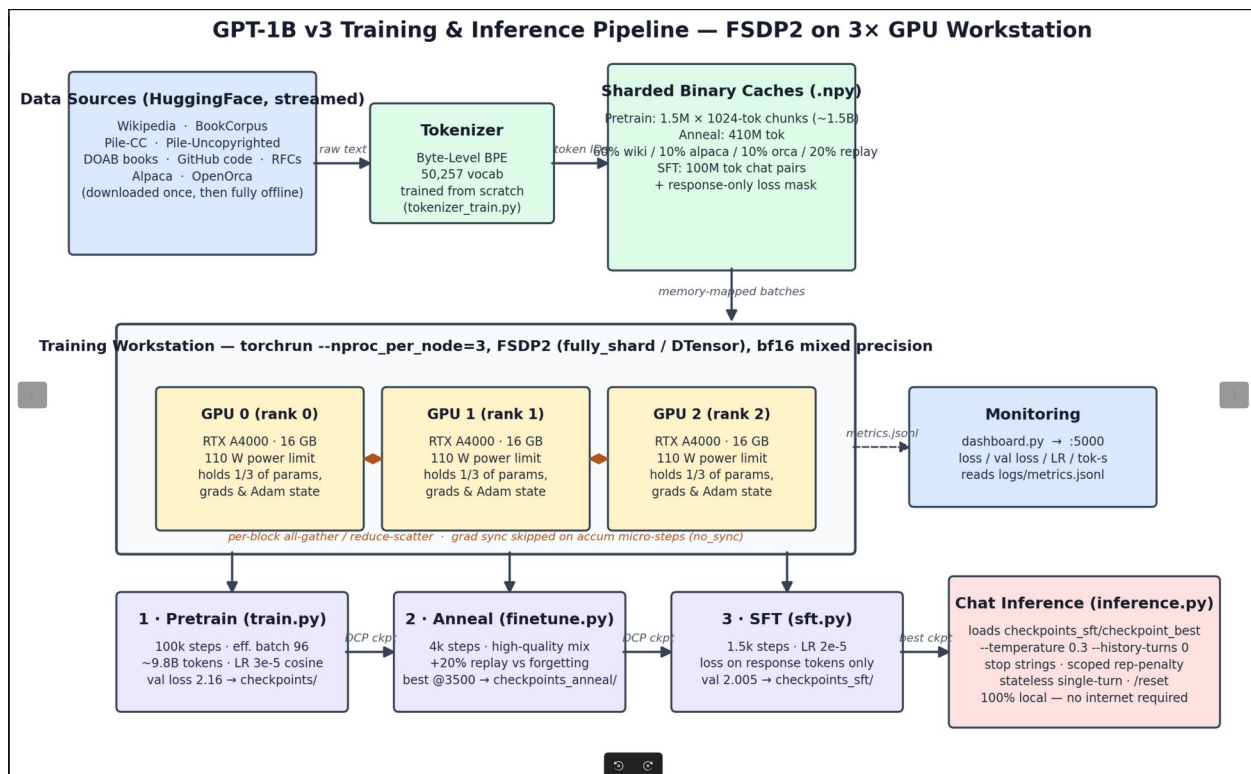
Why Train Locally:

Locally hosted AI keeps private data off third-party servers. Training locally extends that same principle from inference to the model itself: the operator controls exactly what data the model learns from, owns the resulting weights outright, and can run the entire pipeline air-gapped after the initial dataset download. It is also the only way to truly understand the cost structure of modern AI. The gap between a 1B-parameter model trained on ~10 billion tokens and a frontier model trained on trillions of tokens across thousands of GPUs becomes concrete and measurable rather than abstract.

Lab Summary:

In this lab, we trained a 1.01-billion-parameter GPT-style language model from scratch on a 3-GPU workstation (3× NVIDIA RTX A4000, 16 GB VRAM each, power-limited to 110 W per card) and carried it through the full modern training pipeline. We first trained a 50,257-token byte-level BPE tokenizer and built a sharded binary pretraining cache of 1.5 million 1024-token chunks (~1.5 billion tokens) streamed from a mixture of Wikipedia, BookCorpus, Pile-CC, open-license book and code archives, RFCs, and instruction data. Pretraining ran for 100,000 optimizer steps under FSDP2 with an effective batch of 96 sequences (98,304 tokens per step, ~9.8 billion tokens total) at roughly 10,000 tokens/second, finishing with a validation loss of 2.16. We then built a 410-million-token anneal cache (60% Wikipedia, 10% Alpaca, 10% OpenOrca, 20% pretraining replay) and ran a 4,000-step continued-pretraining phase, reaching a validation loss of 2.25 on the harder anneal mixture with 56.5% top-1 next-token accuracy. Finally, we ran 1,500 steps of supervised fine-tuning on a 100-million-token chat-formatted cache (Alpaca + OpenOrca) with a response-only loss mask, producing a best checkpoint at validation loss 2.005. The finished model runs interactively in a chat interface with stop-string handling, properly scoped repetition penalty, and stateless conversation history, and behaves as a fluent, well-mannered assistant within the limits of its scale.

System Diagram:



Steps and Configuration:

Step 1: Train the Tokenizer

Before any training, raw text must be converted into integer tokens. Run `tokenizer_train.py` to train a Byte-Level BPE tokenizer with a target vocabulary of 50,257 tokens on text streamed from the same datasets used for pretraining. The script outputs `vocab.json` and `merges.txt` into the `tokenizer/` directory. Token ID 0 is reserved as the end-of-text (EOS) token.

```
(gpt1b-v2) andy@AI-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$ cd "/home/andy/GPT-1b-v3 Project FSDP2/gpt-1b-v3"
ls -lh tokenizer/
python -c "from tokenizers import ByteLevelBPETokenizer; t = ByteLevelBPETokenizer('tokenizer/vocab.json','tokenizer/merges.txt'); print('Vocab size:', t.get_vocab_size())"
total 1.3M
-rw-r--r-- 1 andy andy 451K May 22 13:51 merges.txt
-rw-r--r-- 1 andy andy 785K May 22 13:51 vocab.json
Vocab size: 50257
(gpt1b-v2) andy@AI-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$
```

Step 2: Build the Pretraining Cache

Tokenizing billions of tokens on the fly would starve the GPUs, so `data.py` streams the dataset mixture (Wikipedia, BookCorpus, Pile-CC, Pile-Uncopyrighted, the Directory of Open Access Books, a GitHub code archive, RFCs, Alpaca, and OpenOrca) from HuggingFace, tokenizes it once, packs it into fixed 1024-token chunks, and writes the chunks to numbered binary `.npy` shards on disk. The full cache is 1,500,000 chunks (~1.5 billion tokens). On every subsequent

run, the dataset memory-maps the shards directly — no re-tokenization. A 2% split is held out for validation.

```
(gpt1b-v2) andy@Ai-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$ python /tmp/cache_demo.py
Cache not found or incomplete - building sharded cache...
Target: 1,500,000 chunks | Shard size: 50,000 chunks/shard
Building cache: 0% | 0/1500000 [00:00<, ?chunk/s]
Streaming from Common Pile (Books)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Building cache: 1% | 0140/1500000 [00:35<17:20, 320.73chunk/s]
```

```
(gpt1b-v2) andy@Ai-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$ ls data_cache/ | head -20
ls data_cache/ | wc -l
du -sh data_cache anneal_cache sft_cache
shard_0000.npy
shard_0001.npy
shard_0002.npy
shard_0003.npy
shard_0004.npy
shard_0005.npy
shard_0006.npy
shard_0007.npy
shard_0008.npy
shard_0009.npy
shard_0010.npy
shard_0011.npy
shard_0012.npy
shard_0013.npy
shard_0014.npy
shard_0015.npy
shard_0016.npy
shard_0017.npy
shard_0018.npy
shard_0019.npy
30
2.9G    data_cache
783M    anneal_cache
294M    sft_cache
```

Step 3: Configure the Model and FSDP2

model.py defines the architecture: d_model=2048, n_layers=18, n_heads=16 (head dimension 128), d_ff=8192, vocab_size=50,257, RoPE positional encoding, no bias terms on the linear layers, and a maximum sequence length of 1024. In train.py, fully_shard() is applied to each TransformerBlock individually and then to the root model, sharding parameters, gradients, and optimizer state across all 3 GPUs as DTensors with bfloat16 mixed precision. RoPE buffers are registered in bfloat16 to match. Checkpoints are saved every few thousand steps with PyTorch Distributed Checkpoint using StateDictOptions(full_state_dict=False), so each rank writes only its own shard and no full-model gather bottleneck occurs on rank 0.

Step 4: Launch Pretraining

Training is launched with torchrun --nproc_per_node=3 train.py via the run_training.sh wrapper script, which auto-restarts the job if it crashes. Because the run takes weeks, the script must be launched detached (setsid nohup ./run_training.sh &) so it survives the SSH session ending. Key

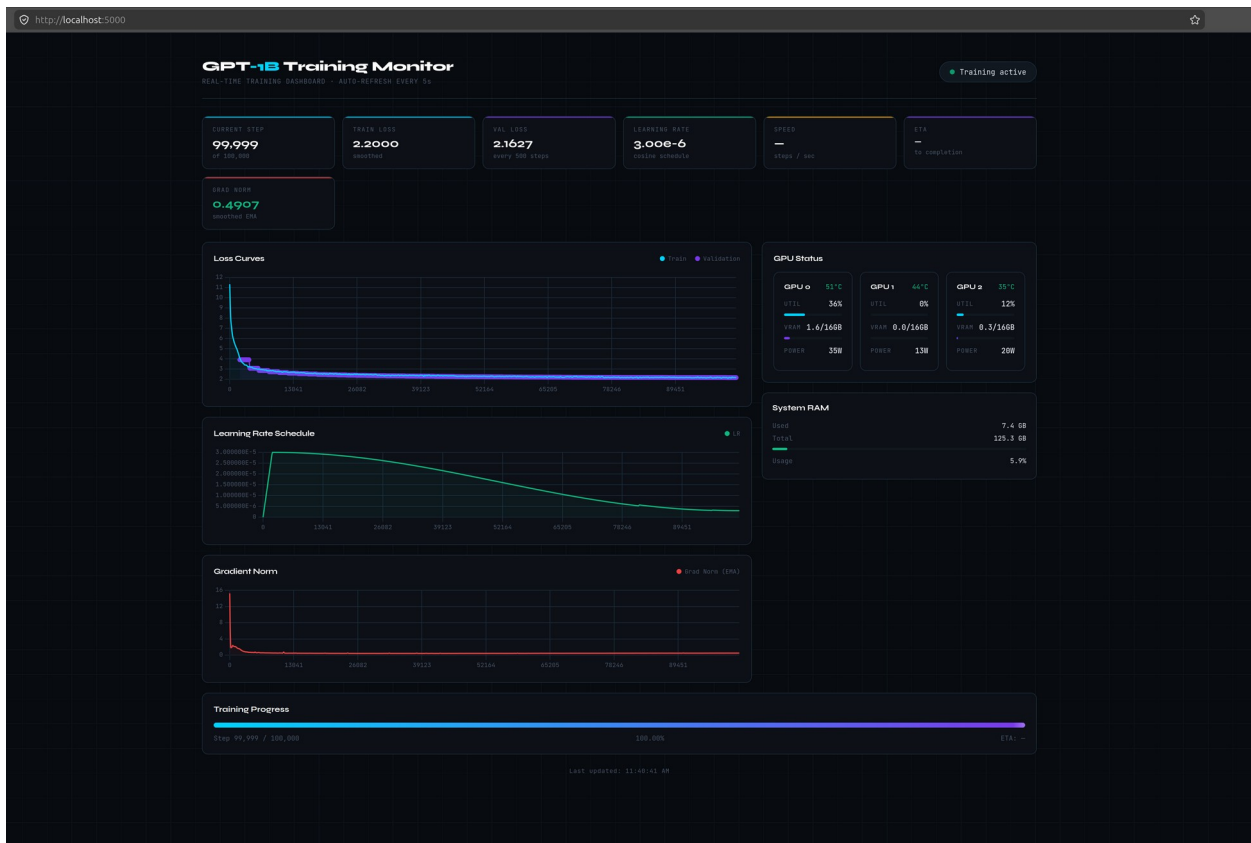
hyperparameters: 100,000 max steps, batch size 8 per GPU $\times$ 3 GPUs $\times$ 4 gradient-accumulation steps = 96 effective batch (98,304 tokens per optimizer step), peak learning rate $3e-5$ with 2,000-step linear warmup and cosine decay to $3e-6$, and AdamW with fused kernels. Gradient sync is suppressed with no_sync semantics on the first three micro-steps of each accumulation cycle. Throughput was approximately 10,100 tokens/second across the three power-limited A4000s, and metrics (loss, validation loss, learning rate, tokens/sec, gradient norm) are appended to logs/metrics.jsonl every 50 steps.

```
(gpt1b-v2) andy@ai-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$ cd "/home/andy/GPT-1b-v3 Project FSDP2/gpt-1b-v3"
tr '\r' '\n' < logs/run.log | sed -n '198849,198863p'
Training: 82%|██████████| 81700/100000 [234:14:11<143:29:16, 28.23s/step, loss=2.1732, val=2.1776, lr=5.26e-06, gnorm=0.470, tok/s=3,484]

step      : 81700/100000
train loss : 2.1732
val loss   : 2.1776
grad norm  : 0.4696
lr         : 5.26e-06
tokens/sec : 3,484
```

Step 5: Monitor with the Dashboard

In a separate terminal, run `python dashboard.py` and open `http://localhost:5000`. The dashboard reads `metrics.jsonl` live and plots training loss, validation loss, learning rate schedule, and throughput, which makes it easy to verify the loss curve is healthy without interrupting the run.



The dashboard after training was completed.

Step 6: Build the Anneal Cache and Run Continued Pretraining

`build_anneal_cache.py` constructs a 410-million-token cache weighted 60% Wikipedia, 10% Alpaca, 10% OpenOrca, and 20% replay of the original pretraining shards (the replay prevents catastrophic forgetting). `run_finetune.sh` then launches `finetune.py`, which loads the final pretraining checkpoint and continues training for 4,000 steps at a reduced learning rate. The best checkpoint (by validation loss) landed at step 3,500 with val loss 2.248 on the anneal mixture, 9.47 perplexity, and 56.5% top-1 / 74.1% top-5 next-token accuracy as measured by `evaluate.py`.


```

(gpt1b-v2) andy@AI-IntegrationLabTest:~/GPT-1b-v3 Project FSDP2/gpt-1b-v3$ cd "/home/andy/GPT-1b-v3 Project FSDP2/gpt-1b-v3"
cat logs/eval_best.log
=====
GPT-1B Model Evaluation
=====
Building model (2048d / 18L)...
Loading checkpoint: checkpoints_anneal/checkpoint_best
  Loaded in 2.0s (step 3,500, loss 2.9062)
Model: 1.01B params | checkpoint step 3,500

Evaluating validation perplexity...
Cache found - 30 shards ready.
Dataset ready - 1,500,000 chunks loaded (memmap)
Validation set: 30,000 chunks (30.7M tokens)
  batch 25/200 - loss 2.1838, ppl 8.88
  batch 50/200 - loss 2.2004, ppl 9.03
  batch 75/200 - loss 2.1805, ppl 8.85
  batch 100/200 - loss 2.1947, ppl 8.98
  batch 125/200 - loss 2.2185, ppl 9.19
  batch 150/200 - loss 2.2246, ppl 9.25
  batch 175/200 - loss 2.2439, ppl 9.43
  batch 200/200 - loss 2.2479, ppl 9.47

Validation loss      : 2.2479
Perplexity           : 9.47
Top-1 accuracy       : 56.51%
Top-5 accuracy       : 74.13%
Loss by ctx pos      : Q1 2.499 | Q2 2.219 | Q3 2.167 | Q4 2.106
Tokens evaluated     : 1,638,400

```

Step 7: Build the SFT Cache and Fine-Tune into a Chatbot

build_sft_cache.py formats Alpaca and OpenOrca samples into a User:/Assistant: chat template, tokenizes each (prompt, response) pair, and stores a parallel loss mask of 0s over the prompt tokens and 1s over the response tokens (including the EOS token), totaling ~100 million tokens. run_sft.sh launches sft.py, which trains for 1,500 steps at peak learning rate 2e-5 with a 50-step warmup, computing cross-entropy loss only where the mask is 1. The best checkpoint came at roughly step 1,250 with a validation loss of 2.0053 and is saved as checkpoints_sft/checkpoint_best.

Step 8: Run Interactive Chat Inference

inference.py loads the SFT checkpoint and provides an interactive chat loop. The recommended invocation is: python inference.py --chat --temperature 0.3 --history-turns 0. Three decoding controls matter at this model scale: (1) stop strings — generation halts if the model emits "User:", preventing it from fabricating the next turn of the conversation; (2) the repetition penalty is applied only to tokens generated in the current response, not the whole transcript; and (3) conversation history defaults to stateless (0 replayed turns), because the SFT data was single-turn and the model fixates on recent history if prior turns are replayed. A /reset command clears the session.

```

Sample generations (temperature 0.8, top-k 50, top-p 0.95):

>>> 'The capital of France is'
the French Crown.

History
In 1799, Pierre Dallery was elected governor of Auxerre and from 1775 he went to the new regime of his daughter Anne de Cécile and joined the Générale du Mérite in 1799, leaving behind his family who were married until 1802. He left Paris with his
wife Louis, a young son of Jean

>>> 'Once upon a time, there was a'
young woman named Lenny. She had been living in an apartment at the corner of Argyll and Pemberton before she died. In 1754, her body was discovered by Dr. Joseph Smith, the son of Henry Smith and Elizabeth Smith, and after this discovery was giv
en to his brother by Charles Smith.

A memorial monument to Frederick I (1844) is dedicated to

>>> 'def fibonacci(n):'

    if n == 0:
        return False

    for i in range(n+1, 1 + 2):
        if i > 0 or i < 1:
            break

    if pytree.getattribute("dtype"):
        for d in pytree.getattr("dtype", "int64"):
            if d[0] != np.ndarray_e:

>>> 'The Internet Protocol (IP) is'
a well-established protocol used by the United Kingdom to ensure that the computer network of any country maintains its security and privacy. IPv6 addresses are usually reserved for the specific region or local government domain in which computer
networks exist; however, some computer networks are not considered suitable for this purpose due to their high vulnerability to malicious software being executed.

DNS is widely adopted by most countries as

>>> 'Question: What is photosynthesis?'\nanswer:'

The answer to this question is:
Photosynthesis is the process of converting carbon dioxide (CO2) into glucose and oxygen. It is an essential part of a cell's metabolism, which provides energy for cells. The conversion of CO2 into glucose through the process of photosynthesis has
been widely studied by scientists over the past century. However, there are still many controversies concerning its importance in plant life

>>> 'In a shocking turn of events, scientists discovered'
that the human sperm was not fully formed. However, this discovery has been confirmed in animal models and is being actively investigated in various laboratories.

In 2007, a study by the American Academy of Sciences, USA, revealed that the human sperm sample was only 3 mm long and had a diameter of about 1 cm (4x10x12mm). This could be attributed to an increase in

```

Problems and Issues:

The earliest problem occurred during the original pretraining attempts, before the v3 configuration described in this report. The combination of a peak learning rate that was set too high, the older FSDP1 API (FullyShardedDataParallel), and instability in the full-state-dict checkpointing path caused training runs to slowly destabilize: a run would appear healthy for thousands of steps, but as it hit a checkpoint save the loss and gradient norms would spike, and the validation loss eventually diverged to NaN (Not a Number), at which point the run was unrecoverable and all progress was useless. This failure led to a v3 rewrite which required a migration to FSDP2 (fully_shard with DTensor), saving checkpoints as sharded state dicts through PyTorch Distributed Checkpoint instead of gathering the full model onto rank 0 at every save, and a more conservative learning-rate schedule (peak 3e-5 with a 2,000-step linear warmup and cosine decay to 3e-6). After these changes, the full 100,000-step pretraining run trained stably to completion with no divergence.

Several operational issues followed during the v3 run. The first involved keeping the training process alive. A 100,000-step pretraining run takes weeks of continuous compute, but any process launched from a terminal normally runs as a child of that login session so when the terminal window is closed, the SSH connection drops, or the user logs out, Linux sends a hangup signal (SIGHUP) to every process the session owns, immediately killing the training and discarding all progress made since the last checkpoint. Early in the project, training runs were repeatedly lost this way to nothing more than a dropped remote connection. The solution was to launch the run_*.sh wrapper scripts fully detached from the login session using setsid nohup ./run_training.sh &. Each part of this command serves a distinct purpose: nohup ("no hangup") instructs the process to ignore the SIGHUP signal and redirects its console output to a log file, since the terminal it would normally print to will no longer exist; setsid ("set session ID") goes further by starting the process in an entirely new session with no controlling terminal at all, so the operating system never associates it with the login session in the first place; and the

trailing ampersand returns the shell prompt immediately by running the job in the background. The result is a free-standing process owned by the system rather than by any user session — the connection can drop or the workstation can be logged out of, and training continues uninterrupted. The trade-off is that the run must then be monitored through its log files (`tail -f logs/run.log`) or the web dashboard rather than a live terminal, and stopping it requires locating the process ID and terminating it manually, since keyboard interrupts no longer reach it. Additionally, unauthenticated HuggingFace downloads were rate-limited during cache builds, which could stall the data stream until an `HF_TOKEN` environment variable was supplied.

The most instructive problems appeared after SFT, when the chatbot initially produced incoherent, drifting output and held conversations with itself. These symptoms that looked like a failed training run but were three separate inference bugs. First, with no stop strings, the model would generate its answer and then keep going, writing the user's next message and answering it. Second, the repetition penalty was being applied across the entire growing transcript instead of just the current response, which suppressed common words and pushed the model progressively off topic as conversations grew. Third, replaying multi-turn history caused the model to fixate on its own previous answer. An example would be where after answering one question about Paris, every subsequent answer became about Paris. This is because the SFT dataset was single-turn and the model never learned to handle multi-turn context. A related latent bug was injecting the literal string "`<|endoftext|>`" into prompts: it byte-encodes into seven junk tokens rather than the actual EOS token ID 0. Fixing all four issues in `inference.py` transformed the apparent quality of the model without changing a single weight, demonstrating that decoding-layer bugs can make a working model look broken.

Conclusion:

In this lab, we successfully executed the complete modern LLM training pipeline all the way from tokenizer training, FSDP2 distributed pretraining over ~9.8 billion tokens, an annealing phase with replay, response-masked supervised fine-tuning, to corrected chat inference, entirely on three workstation GPUs totaling 48 GB of VRAM. The resulting assistant is fluent and well-mannered: it stays on topic, stops cleanly, and answers common-knowledge questions (geography, science basics, simple writing and advice tasks) competently. Its limits are equally clear: it is factually unreliable on niche topics, cannot perform arithmetic or multi-step reasoning, and fails at code generation. Critically, after the inference fixes, these remaining failures are attributable to model scale and data which is exactly the boundary this lab set out to find.

Throughout LLM training, memory is the main binding constraint in local LLM training, and sharded data parallelism like FSDP2 is what makes a 1B-parameter model trainable on 16 GB cards at all; the techniques used here (sharding, mixed precision, gradient accumulation, distributed checkpointing) are the same ones used at datacenter scale, just on three GPUs instead of thousands. When debugging a "bad" chatbot, inference-level decoding bugs must be separated from model-level capability limits before concluding that retraining is needed: three small fixes in the decoding loop did more for perceived quality than any amount of additional training would have. Building on previous labs, this lab completes the progression from hosting someone else's model to owning every stage of one's own.